

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ  
ВЫСШЕГО ОБРАЗОВАНИЯ  
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

---

Кафедра защиты информации

## ТЕХНИЧЕСКОЕ ЗАДАНИЕ

по расчетно-графической работе  
по дисциплине «Программирование»

Выполнил:

студент группы АБ-522

Никулин Р. Д.

Проверил:

---

(должность)

Архипова А. Б.

Новосибирск 2026

## **АННОТАЦИЯ**

Настоящий документ является техническим заданием на разработку программы «Multi-Algo Cryptotool» и составлен по ГОСТ 19.201-78. Документ выдан кафедрой защиты информации ФГБОУ ВО «Новосибирский государственный технический университет» в качестве задания на расчетно-графическую работу по дисциплине «Программирование».

В документе приведены наименование и область применения программы, основания для разработки, назначение разработки, требования к программе, требования к программной документации, технико-экономические показатели, стадии и этапы разработки, а также порядок контроля и приемки. Текст задания приведен без изменений; переработке подверглось только оформление.

## СОДЕРЖАНИЕ

|     |                                                               |    |
|-----|---------------------------------------------------------------|----|
| 1   | Введение .....                                                | 4  |
| 1.1 | Наименование программы .....                                  | 4  |
| 1.2 | Область применения .....                                      | 4  |
| 2   | Основания для разработки .....                                | 4  |
| 2.1 | Документ, на основании которого ведется разработка .....      | 4  |
| 2.2 | Организация, утвердившая документ .....                       | 4  |
| 2.3 | Тема разработки .....                                         | 4  |
| 3   | Назначение разработки .....                                   | 4  |
| 3.1 | Функциональное назначение .....                               | 4  |
| 3.2 | Эксплуатационное назначение .....                             | 5  |
| 4   | Требования к программе или программному изделию .....         | 5  |
| 4.1 | Требования к функциональным характеристикам .....             | 5  |
| 4.2 | Требования к надежности .....                                 | 9  |
| 4.3 | Требования к составу и параметрам технических средств .....   | 10 |
| 4.4 | Требования к информационной и программной совместимости ..... | 10 |
| 5   | Требования к программной документации .....                   | 18 |
| 5.1 | Перечень сдаваемых документов .....                           | 18 |
| 5.2 | Требования к содержанию Пояснительной записки .....           | 18 |
| 6   | Технико-экономические показатели .....                        | 19 |
| 7   | Стадии и этапы разработки .....                               | 19 |
| 8   | Порядок контроля и приемки .....                              | 19 |
| 9   | Список использованных источников .....                        | 20 |

# **1 ВВЕДЕНИЕ**

## **1.1 Наименование программы**

Программа «Multi-Algo Cryptotool».

## **1.2 Область применения**

Программа предназначена для использования в учебных и демонстрационных целях для наглядного изучения и применения различных криптографических алгоритмов. Приложение позволяет пользователям шифровать и расшифровывать текст и любые файлы с использованием криптографических алгоритмов, динамически подключаемых из библиотек.

# **2 ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ**

## **2.1 Документ, на основании которого ведется разработка**

Документ, на основании которого ведется разработка — задание на расчетно-графическую работу по дисциплине «Программирование».

## **2.2 Организация, утвердившая документ**

Документ утвержден кафедрой «Защиты информации» ФГБОУ ВО «Новосибирский государственный технический университет».

## **2.3 Тема разработки**

Тема разработки — «Разработка консольного приложения с модульной архитектурой для шифрования и расшифрования информации».

# **3 НАЗНАЧЕНИЕ РАЗРАБОТКИ**

## **3.1 Функциональное назначение**

Программа должна обеспечивать возможность криптографического преобразования данных с выбором алгоритма и ключа шифрования пользователем во время запуска программы.

### **3.2 Эксплуатационное назначение**

Программа предназначена для использования студентами и преподавателями в среде операционных систем на базе ядра Linux и ОС семейства Windows через интерфейс командной строки.

## **4 ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ**

### **4.1 Требования к функциональным характеристикам**

#### **4.1.1 Состав выполняемых функций**

Программа должна обеспечивать выполнение следующих функций посредством аргументов командной строки:

- вывод справки о программе с перечислением и описанием всех аргументов командной строки;
- выбор алгоритма шифрования;
- выбор режима работы: шифрование, расшифрование или генерация ключа посредством криптостойкого генератора псевдослучайных чисел для выбранного алгоритма шифрования;
- управление ключом: чтение ключа из файла, или из стандартного потока ввода, или генерация ключа;
- управление входными данными: чтение данных из указанного файла или из стандартного потока ввода;
- управление выходными данными: запись результата в указанный файл или в стандартный поток вывода;
- сохранение ключа в файл или в стандартный поток вывода.

#### **4.1.2 Требования к организации и представлению данных**

##### **4.1.2.1 Универсальность форматов**

Программа должна обеспечивать корректную обработку файлов любых типов (текстовые, исполняемые, мультимедиа) без искажения содержимого.

#### **4.1.2.2 Внутреннее представление**

Для обеспечения универсальности, все обрабатываемые данные внутри программы должны храниться и передаваться в виде массивов байт. Использование строковых или иных типов для хранения данных (ключей, шифротекстов) не допускается.

#### **4.1.2.3 Режим работы файлов и стандартных потоков**

- чтение и запись файлов должны осуществляться с помощью файловых потоков в бинарном режиме;
- стандартные потоки ввода-вывода должны использоваться в бинарном режиме.

#### **4.1.2.4 Обработка данных произвольной длины**

В случае реализации блочных алгоритмов шифрования следует корректно обрабатывать входные данные, длина которых не кратна размеру блока шифрования, используя стандарт выравнивания PKCS#7.

#### **4.1.2.5 Формат криптографических ключей**

Генерация, чтение, сохранение и передача ключей шифрования должны осуществляться в бинарном формате (в виде массива байт).

#### **4.1.3 Требования к режимам функционирования**

Программа должна обеспечивать работу в трех взаимоисключающих режимах в зависимости от переданных аргументов командной строки.

##### **4.1.3.1 Информационно-справочный режим:**

- условие запуска: отсутствие аргументов командной строки или наличие флага вызова справки;
- функция: вывод в стандартный поток вывода краткой справки о синтаксисе программы, списке поддерживаемых алгоритмов и доступных флагах;
- результат: успешное завершение работы программы.

#### **4.1.3.2 Режим генерации ключа:**

- условие запуска: наличие флага режима работы с выбранным режимом — генерация ключа;
- функция: генерация ключа с помощью криптостойкого генератора псевдослучайных чисел для выбранного алгоритма;
- вывод данных: сохранение ключа в файл или в поток вывода согласно значению соответствующего флага командной строки.

#### **4.1.3.3 Режим криптографического преобразования данных:**

- условие запуска: наличие полного набора обязательных аргументов, с помощью которых программа однозначно определит алгоритм шифрования, режим работы, источник входных данных и место вывода выходных данных, а также при условии отсутствия конфликтов использования стандартных потоков ввода-вывода;
- функция: динамическая загрузка библиотеки выбранного алгоритма, выполнение операции шифрования или расшифрования потока данных;
- результат: формирование выходного потока данных, опционально сохранение ключа в файл или в стандартный поток вывода, завершение работы.

#### **4.1.4 Требования к логическому контролю аргументов**

Программа должна реализовывать валидацию комбинаций аргументов:

- при попытке запуска режима криптографического преобразования данных без указания обязательных параметров, программа должна выводить информативное сообщение об ошибке и завершать работу;
- при указании конфликтующих флагов приоритет отдается режиму вывода справки.

## 4.1.5 Требования к интерфейсу пользователя

### 4.1.5.1 Тип интерфейса

Взаимодействие с пользователем осуществляется через интерфейс командной строки. Примеры взаимодействия пользователя с программой приведены на рисунках 4.1–4.4.

```
cryptum --help
```

Рисунок 4.1 — Пример команды вызова справки

```
cryptum --algorithm aes --mode generate-key --save-key key.bin
cryptum --algorithm chacha20 --mode generate-key --write-key |
base64 > key.txt
```

Рисунок 4.2 — Примеры команд генерации ключа

```
cryptum --algorithm chacha20 --mode encrypt --key key.bin
--input report.pdf --output report.pdf.enc
tar -czf - /etc | cryptum -a aes -m encrypt -k backup.key -o
etc_backup.tar.gz.enc
cryptum --algorithm chacha20 --mode encrypt --generate-key
--save-key session.key --input lecture.mp4 | nc arhipova.ab
1337
```

Рисунок 4.3 — Примеры команд шифрования

```
cat backup.tar.gz.enc | cryptum --algorithm aes --mode decrypt
--key backup.key | tar -xzf - -C /restore
cryptum -a chacha20 -m decrypt -k <(base64 -d key.txt) -i
data.enc -o data.bin
cat secret.txt.enc | cryptum --algorithm chacha20 --mode decrypt
--key secret.key | less
```

Рисунок 4.4 — Пример команд расшифрования

### 4.1.5.2 Требования к синтаксису аргументов командной строки

Программа должна поддерживать стандартный синтаксис аргументов GNU. Для каждого аргумента командной строки должен быть предусмотрен как короткий, так и длинный псевдоним:

- короткий псевдоним состоит из одного дефиса и одной латинской буквы, пример показан на рисунке 4.5;

- длинный псевдоним состоит из двух дефисов и полного человекочитаемого слова, пример показан на рисунке 4.6.

`-a chacha20`

Рисунок 4.5 — Пример короткого псевдонима аргумента командной строки

`--algorithm chacha20`

Рисунок 4.6 — Пример длинного псевдонима аргумента командной строки

## **4.1.6 Требования к локализации и кодировке текстовых данных**

### **4.1.6.1 Язык интерфейса**

Программа должна поддерживать вывод сообщений на английском и/или русском языке.

### **4.1.6.2 Кодировка текстовых данных**

При взаимодействии программы с командной строкой все текстовые данные должны обрабатываться в кодировке UTF-8 для обеспечения корректного отображения символов на различных операционных системах.

Программа должна автоматически настраивать кодовую страницу терминала при запуске под управлением ОС Windows для принудительного использования UTF-8.

## **4.2 Требования к надежности**

Требования к надежности программы:

- устойчивость к ошибкам: программа должна корректно обрабатывать отсутствие файлов, указанных как входные данные, ошибки прав доступа, некорректные ключи и отсутствие необходимых динамических библиотек;
- контроль входных данных: проверка валидности аргументов командной строки перед началом выполнения операций.

- обработка исключений: использование механизмов исключений C++ для предотвращения аварийного завершения работы программы и вывода текста ошибки;

### **4.3 Требования к составу и параметрам технических средств**

#### **4.3.1 Требования к техническим средствам**

##### **4.3.1.1 Процессор**

Программа должна выполнять свои функции в полном объеме на процессорах на архитектуре x86\_64 или AArch64.

##### **4.3.1.2 Оперативная память**

Программа должна работать при не менее 512 Мб свободной оперативной памяти.

##### **4.3.1.3 Дисковая подсистема**

Программа должна работать при не менее 64 Мб свободного места на диске.

#### **4.3.2 Требования к использованию вычислительных ресурсов**

##### **4.3.2.1 Потребление оперативной памяти**

Потребление оперативной памяти при обработке файлов размером до 1 Гб не должно превышать 128 Мб благодаря использованию потоковой обработки/буферизации в тех случаях, когда запись выходных данных осуществляется в файл.

### **4.4 Требования к информационной и программной совместимости**

#### **4.4.1 Операционные системы**

Программа должна работать на ОС на базе ядра Linux версии 5.x и выше, соблюдающих стандарты FHS, и ОС семейства Microsoft Windows версии 10/11 и выше. Разработка ПО должна быть ориентирована на Linux. Допускается реализация программы исключительно под Linux, при

условии обеспечения возможности переноса кода под Windows в будущем без изменения архитектуры приложения.

#### **4.4.2 Язык программирования: C++, стандарт C++17**

#### **4.4.3 Среда сборки: CMake версии не ниже 3.10 или Make версии не ниже 4.2**

#### **4.4.4 Требования к архитектуре и стилю программирования**

##### **4.4.4.1 Модульная архитектура**

Программа должна быть реализовано в соответствии с принципами модульной архитектуры. Главный модуль программы должно быть отделено от реализации конкретных алгоритмов шифрования. Криптографические алгоритмы должны располагаться в своих динамически подключаемых библиотеках.

##### **4.4.4.2 Чистая архитектура**

При проектировании должны соблюдаться принципы разделения ответственности и инверсии зависимостей. Логика работы с файловой системой и пользовательским вводом/выводом должна быть отделена от бизнес-логики.

##### **4.4.4.3 Конвенция именования**

Исходный код должен быть оформлен в едином стиле:

- структуры, перечисления и псевдонимы типов — UpperCamelCase;
- функции и методы — snake\_case;
- переменные — snake\_case;
- константы — UPPER\_CASE;
- пространства имен — snake\_case;
- файлы — snake\_case.

Общие рекомендации:

- избегать сокращений, если они не общеприняты;

- не начинать имена с нижнего подчеркивания, так как это зарезервировано для внутренних нужд компилятора;
- соблюдать выбранный стиль последовательно во всем проекте.

#### **4.4.4.4 Требования к документированию и качеству исходного кода**

Исходный код должен быть написан так, чтобы его логика была понятна читающему код программисту без использования комментариев, за счет грамотного выбора имен переменных, функций и четкой структуры алгоритмов.

Строго запрещается использование длинных, бессмысленных или описывающих очевидные действия построчных комментариев, характерных для систем автоматической генерации кода на базе ИИ.

Комментарии в коде допускаются исключительно для описания неочевидных архитектурных решений, хаков, специфических требований к выравниванию памяти или для ссылок на математические стандарты криптографических алгоритмов.

#### **4.4.5 Требования к парадигме программирования и языковым конструкциям**

##### **4.4.5.1 Парадигма разработки**

Программа должна быть реализована в процедурном стиле. Запрещено использовать полиморфизм для бизнес-логики.

##### **4.4.5.2 Использование стандартной библиотеки**

Требуется использовать стандартные контейнеры C++17. Обработка данных в контейнерах должна осуществляться с помощью итераторов и стандартных алгоритмов.

#### **4.4.6 Установщики и деинсталлеры**

Установщики и деинсталлеры требуют права администратора для копирования файлов в системные директории.

#### 4.4.6.1 Для Linux:

- реализуется как сценарий оболочки Bash;
- создает символическую ссылку или скрипт в директории, которая уже есть в Path.

#### 4.4.6.2 Для Windows:

- реализуется как сценарий Batch или PowerShell;
- обновляет системную переменную окружения Path.

### 4.4.7 Требования к интерфейсам динамических библиотек и управлению памятью

#### 4.4.7.1 Стандартизация бинарного интерфейса

Взаимодействие главного модуля программы с подключаемыми динамическими библиотеками должно осуществляться в соответствии со стандартом C ABI. Для безопасной передачи данных должны использоваться стандартизированные структуры-обертки над указателями и размерами буферов, пример которых приведен на рисунке 4.7.

Спецификация бинарного интерфейса модулей шифрования требует экспорта следующих функций:

- функция получения информации об алгоритме шифрования, в частности длины ключа;
- функция вычисления размера выходного буфера;
- функции шифрования и расшифрования.

Примеры сигнатур функций приведены на рисунках 4.8–4.10.

```
struct ConstBuffer {  
    const uint8_t* data;  
    size_t size;  
};  
struct MutBuffer {  
    uint8_t* data;  
    size_t size;  
};
```

Рисунок 4.7 — Примеры структур-обертки

```
struct AlgorithmInfo {
    const char* algorithm_name;
    size_t key_size;
};
extern "C" const AlgorithmInfo* get_algorithm_info();
```

Рисунок 4.8 — Пример сигнатуры функции получения метаданных

```
extern "C" size_t get_output_size(size_t input_size, int
    operation_type);
```

Рисунок 4.9 — Пример сигнатуры функции вычисления размера выходного буфера

```
extern "C" int encrypt(ConstBuffer key, ConstBuffer input,
    MutBuffer* output);
extern "C" int decrypt(ConstBuffer key, ConstBuffer input,
    MutBuffer* output);
```

Рисунок 4.10 — Пример сигнатур функций шифрования и расшифрования

#### 4.4.7.2 Изоляция управления памятью

Основным принципом безопасного управления памятью в куче при обмене данными является выделение памяти на стороне главного модуля программы. Для получения длины буфера выходных данных у функции шифрования или расшифрования следует использовать функцию вычисления размера выходного буфера.

В случае, если невозможно избежать выделения памяти в куче динамической библиотеки, она должна предоставлять главному модулю программы функцию для безопасного удаления выделенной памяти.

#### 4.4.7.3 Защита данных от утечек из оперативной памяти

Для предотвращения утечки конфиденциальной информации (ключей шифрования, открытого текста) через дамп памяти или файл подкачки, программа должна выполнять принудительную перезапись буферов памяти, содержащих конфиденциальную информацию, перед их освобождением. Очистка буферов памяти не должна быть удалена компилятором

вследствие оптимизации при компиляции. Разрешается использовать платформозависимые функции ОС для очистки памяти.

#### **4.4.7.4 Требования к обработке ошибок в динамических библиотеках**

Функции динамической библиотеки не должны выбрасывать исключения, выходящие за границы библиотеки. В случае возникновения ошибки в функции шифрования/расшифрования должен возвращаться ненулевой код ошибки, а главный модуль программы должен корректно обработать его и вывести сообщение об ошибке.

#### **4.4.7.5 Требования к интерфейсу взаимодействия с динамическими библиотеками**

Для обеспечения безопасной и предсказуемой работы, а также заменяемости динамических библиотек, главный модуль программы не должен делать предположений о характеристиках загруженного алгоритма. Каждая динамическая библиотека обязана предоставлять функцию получения метаданных для получения информации о названии алгоритма шифрования и о длине ключа.

#### **4.4.7.6 Требования к вектору инициализации**

Если алгоритм шифрования использует вектор инициализации, то при выполнении операции шифрования он должен генерироваться внутри динамической библиотеки выбранного алгоритма с использованием криптостойкого генератора псевдослучайных чисел.

Сгенерированный вектор инициализации должен автоматически помещаться в начало шифротекста. При выполнении операции расшифрования следует извлекать вектор инициализации из начала шифротекста. Главный модуль программы не должен участвовать в генерации, хранении или передаче вектора инициализации.

#### **4.4.7.7 Требования к загрузке динамических библиотек**

Перечень поддерживаемых алгоритмов шифрования и базовые имена соответствующих им динамических библиотек должны быть заданы в исходном коде основного исполняемого модуля программы.

Основной модуль должен автоматически формировать платформозависимое имя файла библиотеки для загрузки во время выполнения в зависимости от целевой операционной системы. Например, при выборе алгоритма `des` в ОС семейства Windows программа должна загружать библиотеку `des.dll`, а в ОС на базе ядра Linux — `libdes.so`. Дополнительные конфигурационные файлы для поиска библиотек не применяются.

#### **4.4.8 Требования к средствам разработки и сборки**

##### **4.4.8.1 Поддерживаемые компиляторы**

Исходный код должен корректно компилироваться следующими инструментами:

- Linux: GCC версии 9.1 и выше или Clang версии 10.0 и выше;
- Windows: MSVC версии 19.20 и выше или MinGW-w64.

##### **4.4.8.2 Параметры компиляции**

Сценарии сборки CMake/Make должны обеспечивать строгий контроль качества кода за счет использования специальных флагов компилятора:

- стандарт языка: требуется соответствие кода стандарту C++17;
- обработка предупреждений: режим, при котором все предупреждения компилятора интерпретируются как ошибки;
- уровень предупреждений: максимальный уровень диагностики.

##### **4.4.8.3 Требования к средствам автоматизации сборки**

###### **4.4.8.3.1 Система CMake:**

- наличие в корневом каталоге проекта конфигурационного файла CMake;

- Обеспечение поддержки генерации нативных файлов сборки для целевой платформы.

#### **4.4.8.3.2 Система Make:**

- наличие в корневом каталоге проекта конфигурационного файла Make;
- сценарий должен содержать правила для компиляции всех модулей и очистки скомпилированных файлов проекта.

### **4.4.9 Требования к организации коллективной разработки**

#### **4.4.9.1 Система контроля версий**

Разработка программного продукта должна вестись с использованием распределенной системы контроля версий Git. Все исходные коды, сценарии сборки и документация должны храниться в едином удаленном репозитории.

#### **4.4.9.2 Стратегия ветвления**

Для обеспечения целостности кода должна применяться стратегия ветвления:

- основная ветка содержит только стабильную, протестированную версию продукта;
- разработка новых функций и исправление ошибок ведутся в отдельных ветках, создаваемых от основной ветки;
- принудительная перезапись в основную ветку запрещена.

### **4.4.10 Требования к тестированию криптографических функций**

#### **4.4.10.1 Отдельная утилита для тестирования**

В составе исходного кода проекта должна быть предусмотрена компиляция отдельного исполняемого файла, предназначенного исключительно для тестирования криптографических алгоритмов в динамических библиотеках.

#### **4.4.10.2 Детерминированность тестов**

Для проверки корректности алгоритмов в ходе тестирования следует фиксировать все параметры криптографической операции. В случае, если алгоритм шифрования использует вектор инициализации, в его динамической библиотеке следует реализовать отдельную функцию шифрования, которая будет позволять задавать вектор вручную. Эта функция должна использоваться только для тестирования.

#### **4.4.10.3 Требования к тестированию**

Для каждого реализованного криптографического алгоритма должно быть разработано и автоматизировано не менее трех тестов на разных входных данных. Разрешается использовать генераторы псевдослучайных чисел для создания данных.

В ходе каждого теста нужно зашифровать данные, затем расшифровать и сравнить расшифрованный текст с исходным.

### **5 ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ**

#### **5.1 Перечень сдаваемых документов**

1. Техническое задание (ГОСТ 19.201-78).
2. Пояснительная записка (ГОСТ 19.404-79).
3. Текст программы (ГОСТ 19.401-78).
4. Руководство пользователя (ГОСТ 19.505-79).

#### **5.2 Требования к содержанию Пояснительной записки**

В разделе «Описание программы» Пояснительной записки (ГОСТ 19.404-79) должно быть представлено графическое изображение (схема) архитектуры программного средства. Схема должна наглядно демонстрировать:

- разделение на основной исполняемый модуль и динамические библиотеки;

- направления потоков данных (ввод-вывод);
- точки вызова функций из динамических библиотек.

## **6 ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ**

Расчет экономической эффективности не производится ввиду учебного характера работы.

## **7 СТАДИИ И ЭТАПЫ РАЗРАБОТКИ**

Разработка программного средства будет осуществляться в соответствии с процессами жизненного цикла, рекомендованными ГОСТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств», и включает следующие этапы:

- Анализ требований.
- Проектирование архитектуры приложения.
- Реализация главного модуля программы.
- Реализация динамических библиотек с алгоритмами шифрования.
- Отладка и тестирование.
- Подготовка отчета.

## **8 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ**

1. Готовый отчет, оформленный в соответствии с установленными требованиями, загружается в DiSpace 2.0 в курсе «Программирование» в ответ на задание РГР.
2. После получения положительного вердикта по отчету, приложение демонстрируется преподавателю. Приемка осуществляется на основании соответствия приложения требованиям настоящего технического задания (ГОСТ 19.201-78).

## **9 СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

- ГОСТ 19.101-77 ЕСПД. Виды программ и программных документов.
- ГОСТ 19.201-78 ЕСПД. Техническое задание. Требования к содержанию и оформлению.
- ГОСТ 56939-2024 Защита информации. Разработка безопасного программного обеспечения. Общие требования.
- ГОСТ Р ИСО/МЭК 12207-2010 Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств.
- ГОСТ Р ИСО/МЭК 25010-2015 Информационная технология. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов.